

可視化・可聴化する通信

情報工学科 2 年
24G11051

久我 丈

—— チーム 30 ——

金枝 優介 (24G1040) 久我 丈 (24G1051)
沢田 迅 (24G1068) 山口 侑希 (24G1136)

2025 年 7 月 10 日

1 はじめに

地震などの広域災害が起こると、放送や通信に障害が発生することがある。総務省によると、2016 年の熊本地震では 2000 以上、2024 年の能登半島地震では 1500 もの固定通信回線にサービス障害の影響が出た [1][2]。また、地下施設や山間部、医療施設、飛行機内といった場所ではそもそも電波通信が困難であったり制限されていたりする事がある。

一般的に利用されている電波通信ではどこで何が起きているか直感的に把握しづらく、このような災害や地理による通信障害が起こっても通信可能な手段を確保することで、被災地への迅速な対応や快適な通信の利用が可能になる。この問題を解決する既存の研究として、可視光を用いた Li-Fi[3] や音楽に情報を乗せる携帯電話端末で受信する音響 OFDM[4]、振動モータによって通信する Vinteraction[5] といった技術が存在する。いずれも電波を使用しない技術であり、LED 照明が普及している昨今、人間の目で知覚できない速度で LED を点滅させて通信する LiFi は街灯や自動車のヘッドライトを利用して通信するなど、既存のインフラに幅広く応用することができる。また、2021 年に日本のスマートフォン普及率は 8 割を超え [6]、音響 OFDM と Vinteraction は携帯端末を利用して誰もが聴覚、触覚的に情報を伝送することが可能である。

本報告書ではこれらの技術を参考に、通信を可視化、可聴化することを目的に Arduino を用いて光、音、振動による通信システムを開発した。具体的には、PC からのテキスト入力により文字を 4 進数変換し、LED で 4 進数を送信した。それを照度センサで取得し、4 進数のシンボルごとに割り当てた周波数の音を出すことで文字の伝送をした。この音をマイクで集音し 2 進数に変換したのち、振動モータを用いて振動の有無で 0, 1 を表現し伝送した。この振動を加速度センサで受け取り、受け取った 2 進数の数列から文字をデコードして光、音、振動を介して通信を行った。それぞれの通信を通信精度と通信速度で評価・考察し、全体のシステムとして既存の技術に比べ有効であるかを明らかにする。

2 作成したシステムの概要

2.1 LED を用いた光通信

2.1.1 システムの概要

光通信の構成図と動作の流れを図 1 に示す。Arduino IDE のシリアルモニタに文字を入力し、7 bit ASCII コードから 4 進数にエンコードする。同期 LED の光を照度センサ（フォトトランジスタ）で検知すると通信を開始する。この LED は通信中常に光っている。変換した数字を 4 つの LED に割り当て、LED を光らせることで 0, 1, 2, 3 からなる数列を表現する。通信の開始と終了については、同期専用の LED を用いて、これが光っている間通信するものとする。受信側の Arduino に接続した照度センサを各 LED の正面に配置し、各照度センサが光を受け取ったら対応する数字を

配列に格納する。同期 LED の光を受け取らなくなったら通信を終了する。

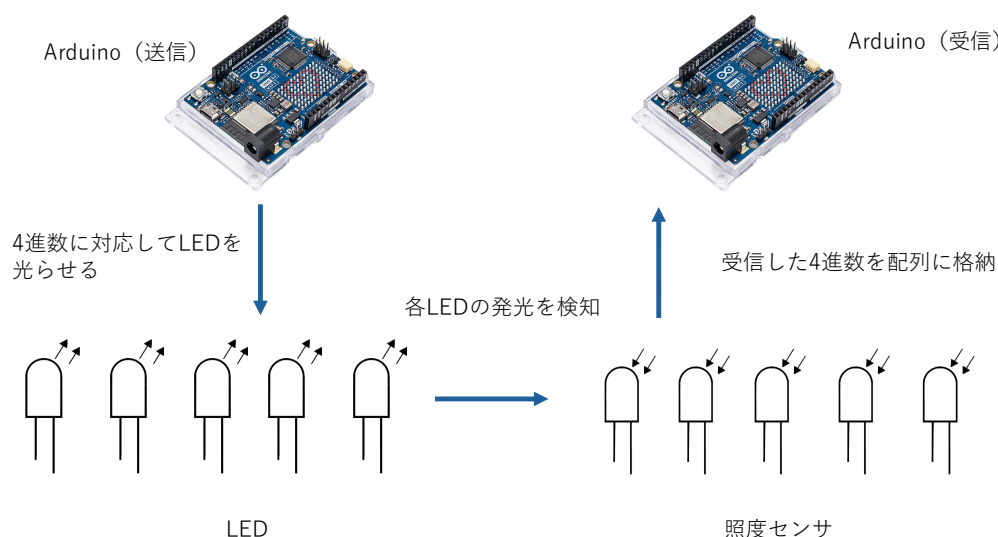


図1 光通信の構成図と動作

2.2 可聴音を用いた音通信

2.2.1 システムの概要

音通信の構成図と動作の流れを図2に示す。通信開始は、同期専用の周波数を出力することで判断する。光を受信して受け取った4進数の数列を2桁1シンボルに区切り、各シンボルに対応させた周波数の音をスピーカーから出力する。それぞれの周波数をマイクで受け取ると、対応した文字列を4進数の数列に変換する。同期周波数をもう一度検出すると通信を終了する。

2.2.2 出力する周波数とその検出

送信側において、受け取った4進数の2桁と周波数と同期周波数の対応を表1に示す。構想段階では同期周波数は1950 Hzだったが、環境音によって意図せず通信を開始、終了することがあったため、環境の影響を受けにくい3.6 kHzに変更した。

音と音の間に無音の時間（ガードインターバル：GI）を設ける。これは隣り合うシンボルの干渉（シンボル間干渉：ISI）を軽減、防止するもので、GIをシンボル長の25%にすることで最低限の時間でISIを防ぎ、効率のよい通信が可能となる[7]。実際には、後述の4.2節で示す目標値の10 bpsに届くよう出力する音の長さを300 ms、GIを音長の25%に設定した。

受信側において、周波数の検出はarduinoFFTライブラリ（<https://github.com/kosme/arduinoFFT>）を用いて行った。アナログ信号をデジタル信号にサンプリングする際、アナログ信号の情報を失

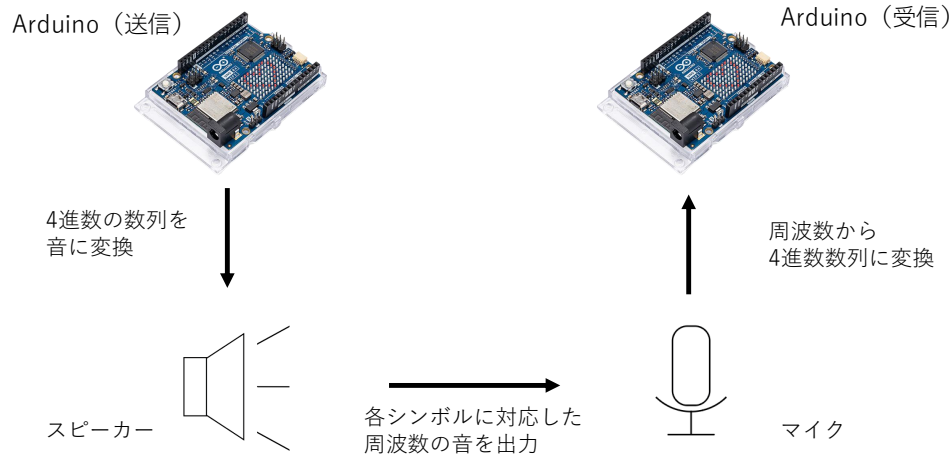


図2 音通信の構成図と動作

わないようにする必要がある [8]. このためには, アナログ信号の周波数を F_m , サンプリング周波数を F_s として式 1 で有る必要がある.

$$F_s > 2F_m \quad (1)$$

このときの F_s をナイキスト周波数という. 本システムで使用する最大周波数は 3.6 kHz であり, ナイキスト周波数は 7.2 kHz となる. また, FFT での周波数分解能はサンプル数を N として F_s/N で表される. 各周波数は 100 Hz 間隔で設定しており, これらを判別するには閾値を ± 50 Hz 未満にする必要がある. 以上を満たすように, 実際にはサンプリング周波数を 8.0 kHz, サンプル数を 256 に設定した.

2.3 振動モータを用いた振動通信

2.3.1 システムの概要

振動通信の構成図と動作の流れを図 3 に示す. 振動が 7 回連続したら通信を開始する. 音通信で受信した 4 進数の数列を 7 bit の ASCII コードに変換する. 変換したら各桁が 0 だったら振動をオフ, 1 だったら振動をオンにすることによって数列を送信する. 加速度センサによって 3 軸の加速度のベクトル距離を検知し, これが閾値を超えたら振動を検出する. この振動の有無の検出により, 振動を 2 進数に変換し, ASCII コードを用いて具体的な文字にデコードする.

表 1 受け取るシンボルと周波数の対応

シンボル (2 桁)	周波数 [kHz]
同期周波数	3.6
00	2.0
01	2.1
02	2.2
03	2.3
10	2.4
11	2.5
12	2.6
13	2.7
20	2.8
21	2.9
22	3.0
23	3.1
30	3.2
31	3.3
32	3.4
33	3.5

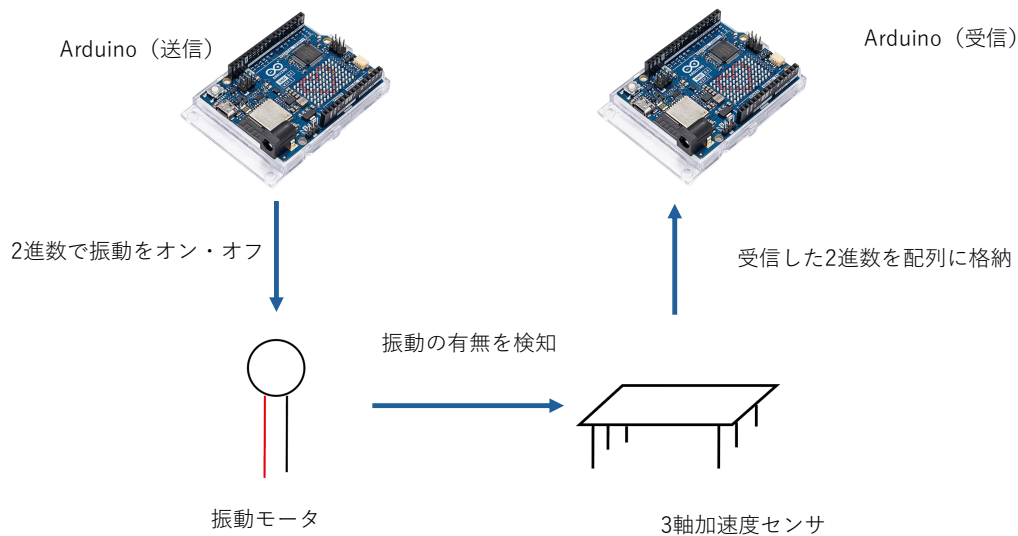


図 3 振動通信の構成図と動作

2.4 システムの実装

2.4.1 使用した機器

本システムに使用した機器を表 2 に示す。

2.4.2 回路図とフローチャート

光の送信，光の受信から音の送信，音の受信から振動の送信，振動の受信をする回路図を図 4，図 6，図 8，図 10 に，フローチャートを図 5，図 7，図 9，図 11 に示す。

2.4.3 主要な関数の機能

本システムにおける Arduino スケッチの主要な関数の機能を説明する。

detectBitByDuration 関数（光受信） 閾値以上の光があると有効としてそのピンに対応するシンボルを保存する。

detectFrequency 関数（音受信） arduinoFFT ライブラリを用いて，FFT でピーク周波数を検出する。

decodeFrequency 関数（音受信） 開始，終了の同期周波数とシンボル周波数から 4 進数 2 桁のデータに変換する。

detectVibration 関数（振動受信） 一定時間内に振動の有無を判断，2 進数データに変換し保存する。

表 2 使用した機器

機器名	仕様	個数
Arduino Uno R4 WiFi	ABX00087	3
ブレッドボード	EIC-102J	3
発光ダイオード		5
フォトトランジスタ	NJL7502L	5
ブレッドボード用ダイナミックスピーカ	MSI28-12R	1
マイクアンプモジュール	AE-MICAMP	1
振動モータ	316040001	1
3 軸加速度センサ	KXR94-2050	1
ダイオード		1
トランジスタ		1
炭素被膜抵抗器	22 Ω	1
炭素皮膜抵抗器	330 Ω	5
炭素皮膜抵抗器	3.3 k Ω	1
hline 炭素皮膜抵抗器	100 k Ω	5

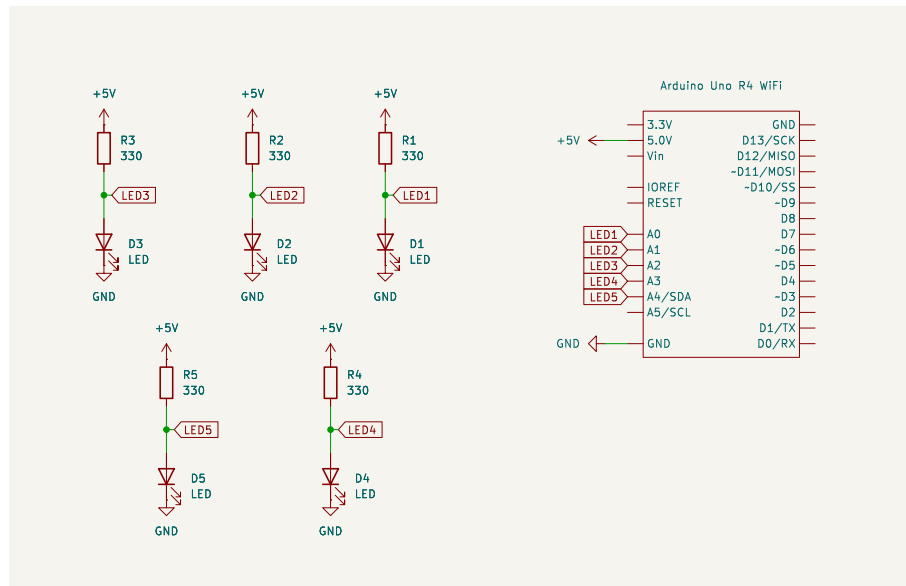


図 4 光送信の回路図

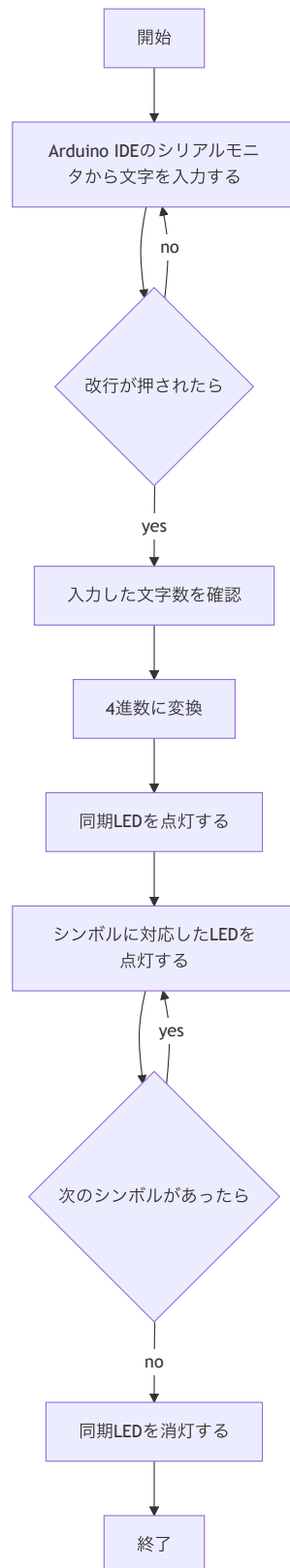


図 5 光送信のフローチャート

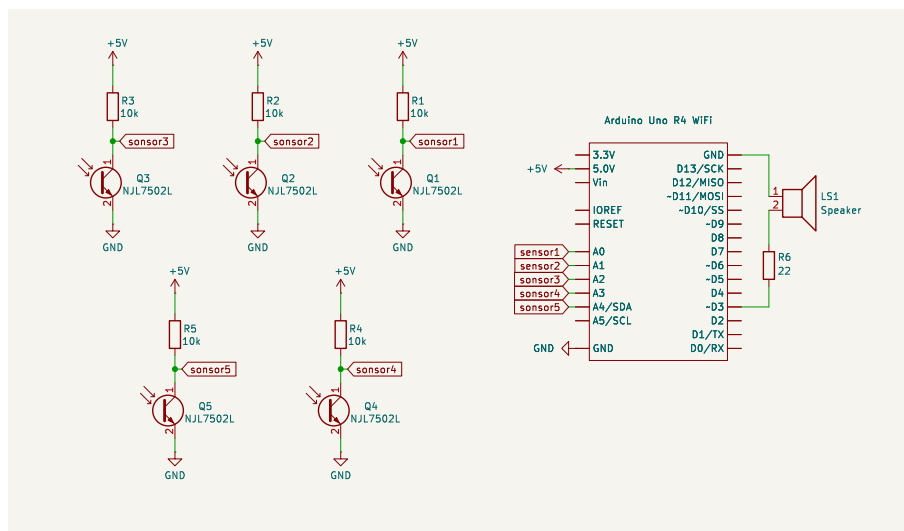


図 6 光受信から音送信の回路図

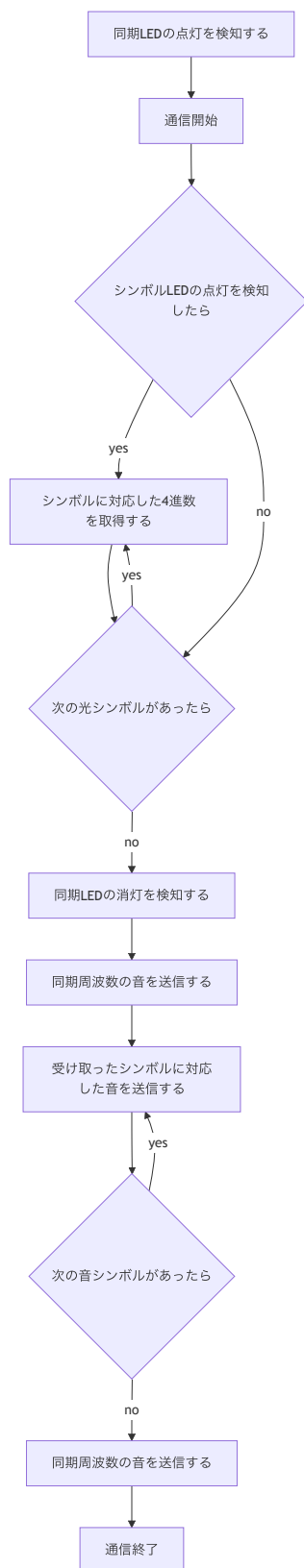


図7 光受信から音送信のフローチャート

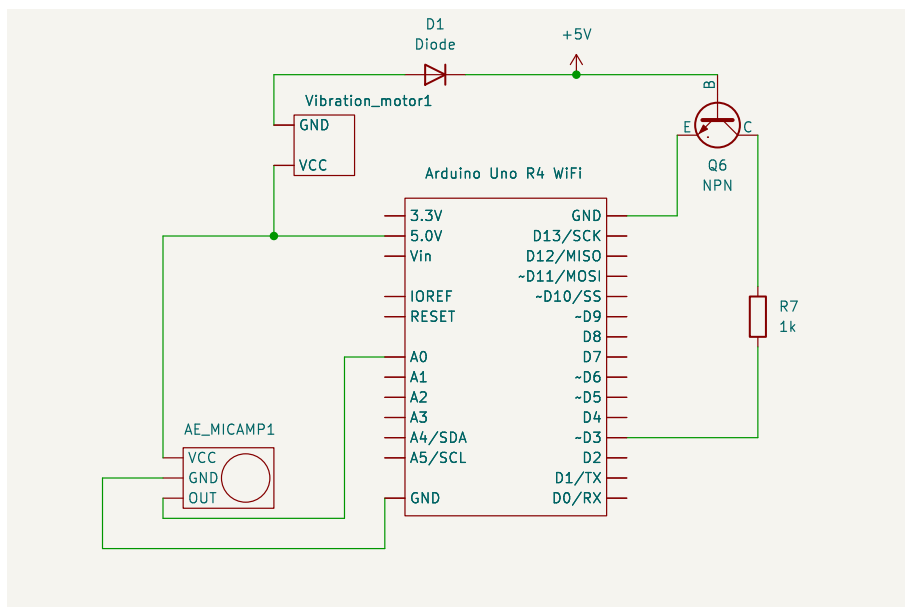


図 8 音受信から振動送信の回路図

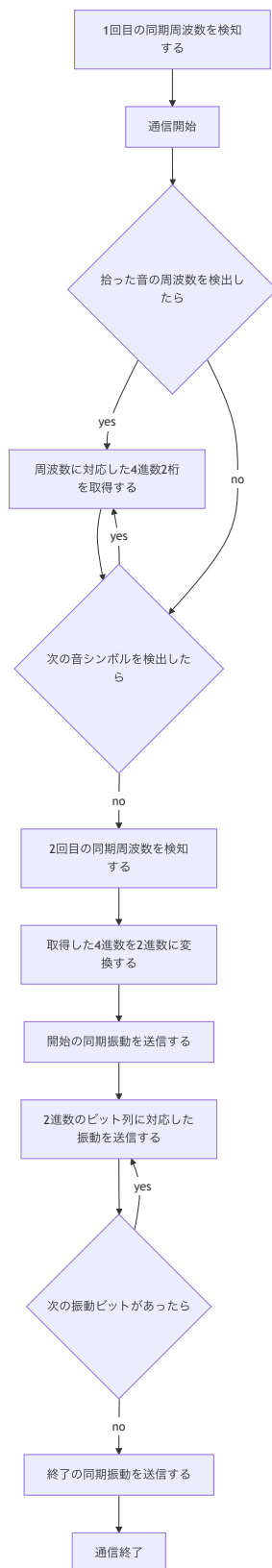


図 9 音受信から振動送信のフローチャート

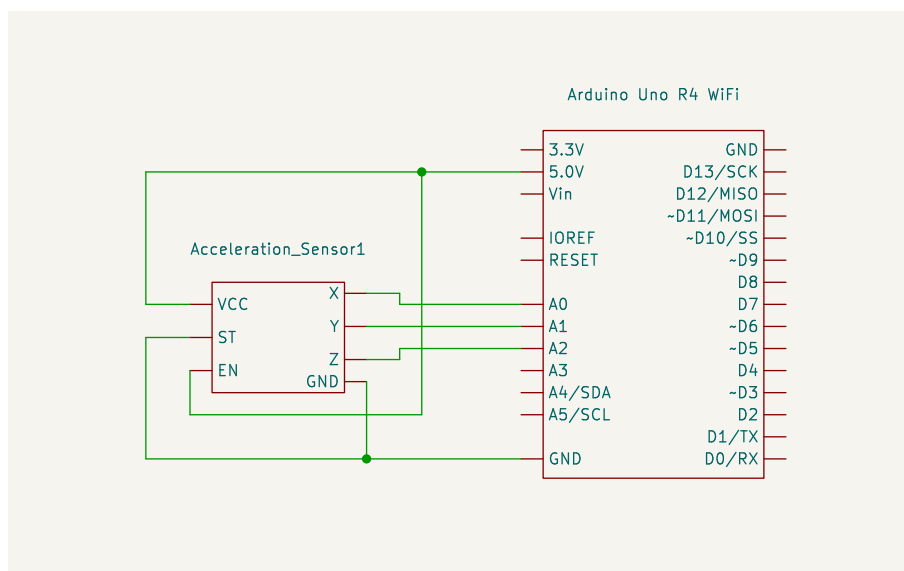


図 10 振動送信の回路図

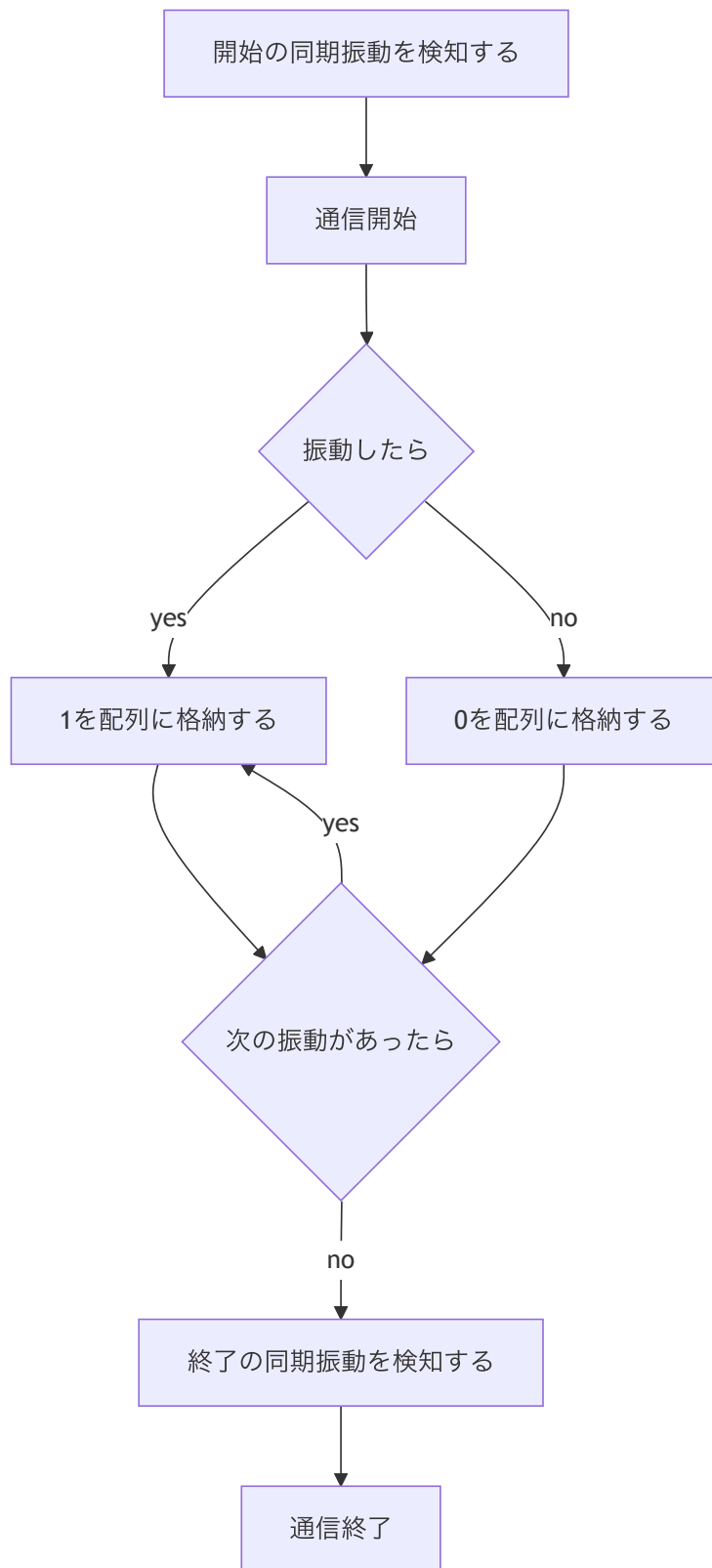


図 11 振動送信のフローチャート

3 スケジュールと担当

本システムでは光通信，音通信，振動通信の3部門に分かれ，それぞれを実装し，結合した．各部門における作業とその担当者，週ごとの作業内容を図12に示す．しかし，実際は音送信のプログラム作成に時間がかかってしまったことと音通信，振動送信が想定より実装が困難であったため担当者を変更し，実際のスケジュールは図13のようになった．報告者は抵抗，スピーカ，マイクの調達，音送受信の回路とプログラムの作成，光通信と音通信，全体のテストを担当した．

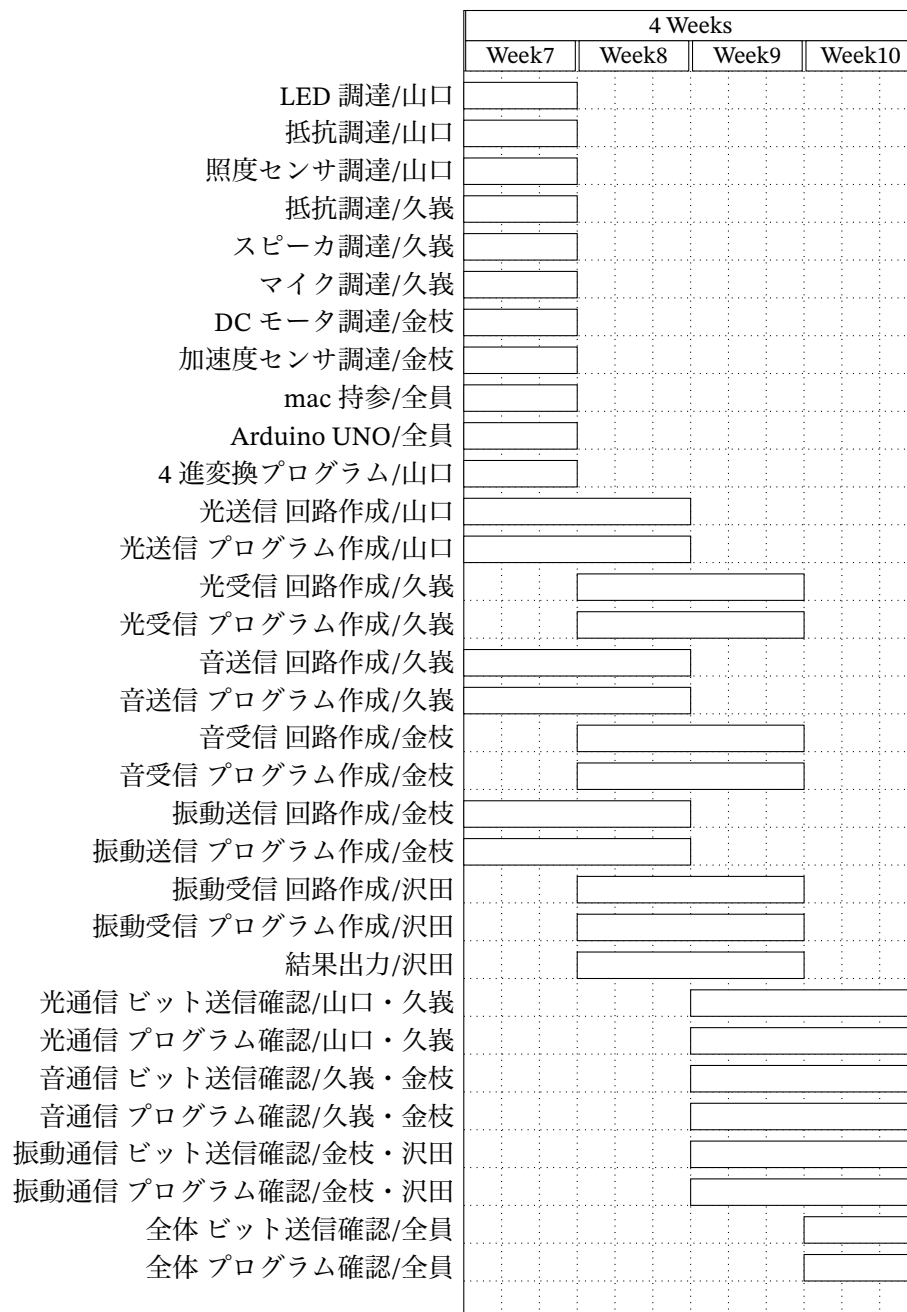


図 12 計画時のスケジュール

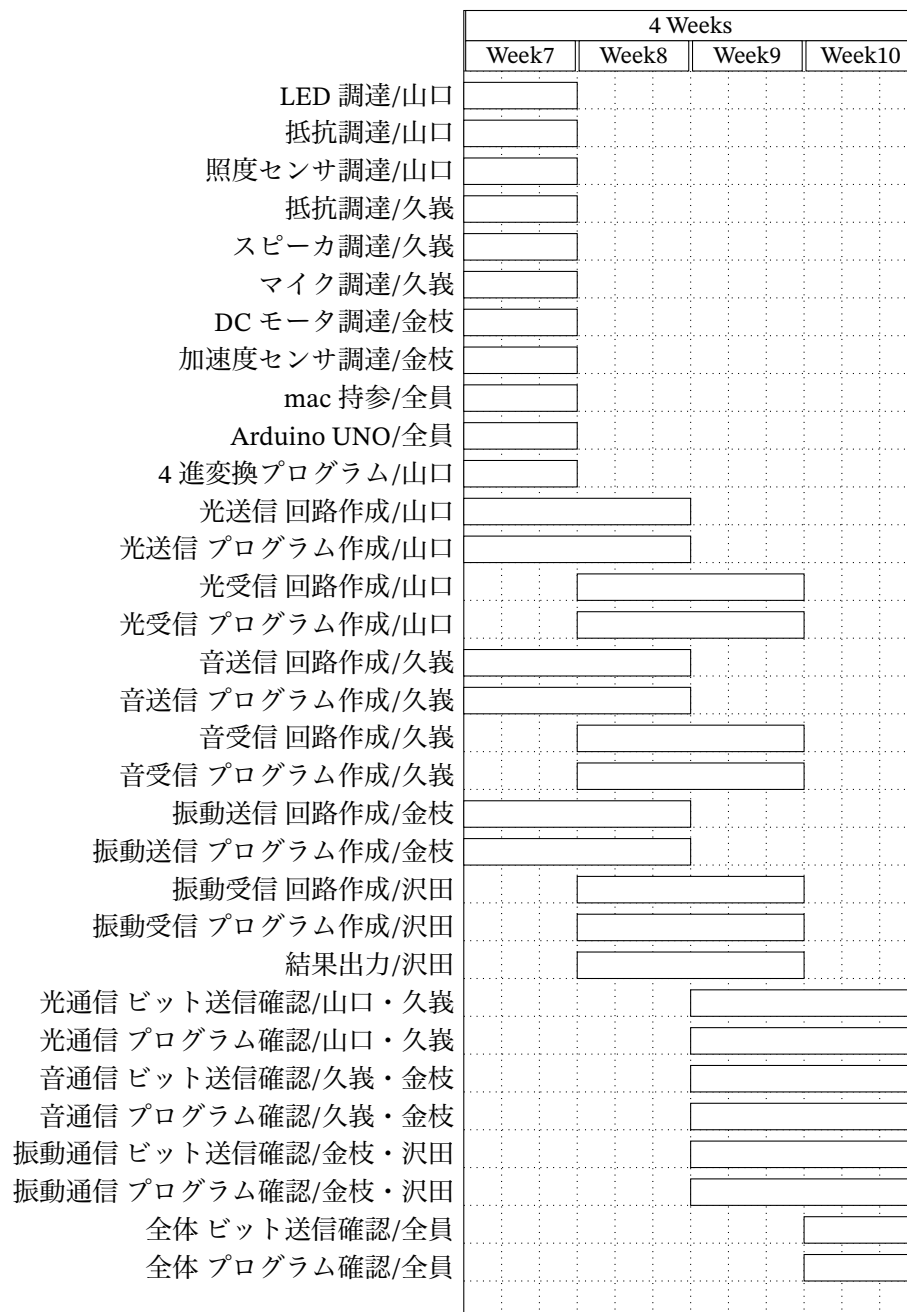


図 13 実際のスケジュール

4 システムの動作

4.1 動作確認の手順

3 部門を結合して動作確認を行った。具体的には、最初にシリアルモニタに"chiba"と入力し、光通信→音通信→振動通信の順で通信を行っていき、各部門にかかる時間と受け取った 4 進数または 2 進数の数列、最終的にデコードされた文字を記録した。動作確認時の条件は以下の通りである。

- 各 Arduino に繋いだ電源はすべて Mac book の USB type-C からのみの給電
- スピーカとマイクの距離は 2 cm
- 振動モータと加速度センサは、4 つのゴムで脚をつくった薄いアクリル板に貼り付け、その距離は 2 cm
- 時間の計測は iphone 14 の「時計」アプリのラップタイムを利用

4.2 目標の設定

本システムが有効であるかの指標として、各部門に表 3 のような目標を設定する。これらの目標に到達することで、有効なシステムであると判断する。bps (bit par second) は、1 秒あたりに送信されたビット数を表す。フレーム成功率 (FSR) は、送信されたフレームの総数に対する誤りがなく通信できたフレームの割合で、総フレーム数を N_a 、誤り無く送信されたフレーム数を N_c として式 2 で表される。ビット誤り率 (BER) は、送信されたビット数に対する誤って受信したビット数の割合で、総ビット数を N_t 、誤りビット数を N_e として、式 3 で表される。

$$FSR = \frac{N_c}{N_a} \quad (2)$$

$$BER = \frac{N_t}{N_e} \quad (3)$$

光通信において、 $FSR \geq 0.90$ 、100 bps 以上であるとシステムが有効であるといえる [9][3]。また、データを音のオンオフで符号化した音響通信の例では、雑音が通信音量以下だと BER は約 2 %、通信音量以上だと BER が 10 %を超えたという報告がある [10]。この研究 [10] における通信速度は 10bps であり、この条件で BER が 2~10 %であるため、10 bps で適切な通信が可能だと考える。これらを踏まえ、 $FSR \geq 0.90$ 、 $BER < 10^{-1}$ 、100 bps 以上（光通信）、10 bps 以上（音、振動通信）を目標として設定する。

4.3 動作結果

計 12 回の試行を行った結果を表 4 に、各部門で変換した 4 進数または 2 進数を表 5、表 6 に示す。光、音、振動通信と全体の経過時間の平均はそれぞれ 1.07 s、5.23 s、14.20 s、20.51 s だった。光通信では一度も間違って通信することはなかったが、音通信では 12 回中 2 回、振動通信では 12

表 3 各部門における到達目標

部門	通信精度	通信速度
光通信	$FSR \geq 0.90$	100 bps 以上
音通信	$BER \geq 10^{-1}$	10 bps 以上
振動通信	$BER \geq 10^{-1}$	10 bps 以上

回中 11 回誤った情報が伝送された。デコード結果は、完全な"chiba"が 1 回、1 文字誤りが 2 回、2 文字誤りが 1 回だった。

表 4 本システムの動作結果

試行回数	光通信の時間 [s]	音通信の時間 [s]	振動通信の時間 [s]	合計時間 [s]	デコード結果
1	1.06	5.28	13.73	20.07	gl{bi
2	1.09	5.21	12.92	19.22	chibq
3	1.09	5.19	13.08	19.36	sl<s0
4	1.08	5.21	20.92	27.21	chija8
5	1.09	5.04	12.26	18.39	gH
6	1.01	5.25	13.08	19.34	cBisA
7	1.05	5.13	15.14	21.32	chkba
8	1.56	4.75	16.50	22.81	oPIRx
9	1.10	6.08	13.11	20.29	sltPp
10	0.98	5.31	12.98	19.27	chiba
11	0.98	5.21	13.23	19.42	sl{'q
12	0.91	5.06	13.49	19.46	cv6&

表 5 光、音通信の変換した 4 進数または 2 進数

試行回数	デコード結果	光受信 (4 進数)	音受信 (2 進数)
1	gl{bi	12031220122112021201	11000111101000110100111000101100001
2	chibq	12031220122112021201	11000111101000110100111000101100001
3	sl<s0	12031220122112021201	11000111101000110100111000101100001
4	chija8	12031220122112021201	11000111101000110100111000101100001
5	gH	12031220122112021201	1100011110100000101100100110
6	cBisA	12031220122112021201	11000111101000110100111000101100001
7	chkba	12031220122112021201	11000111101000110100111000101100001
8	oPIRx	12031220122112021201	11000111101000110100111000101100001
9	sltPp	12031220122112021201	11000111101000110100111000101100001
10	chiba	12031220122112021201	11000111101000110100111000101100001
11	sl{'q	12031220122112021201	11000111101000110100111000101100001
12	cv6&	12031220122112021201	1100011110011000101100100110

表 6 振動通信の変換した 4 進数または 2 進数

試行回数	デコード結果	振動受信 (2 進数)
1	gl{bi	1100111111110011110111100010110100100000000000
2	chibq	記録ミス
3	sl<s0	記録ミス
4	chija8	11000111101000110100111010101100001000000000010000111000 000000010001000000000000
5	gH	11001111001000001011001011100000000000
6	cBisA	1100011100001011010011110011100000100000000000
7	chkba	1100011110100011010111100010110000100100001100000000000
8	oPIRx	11011111010000100100110100101111000000001000100000000000
9	sltPp	11100111111100111010010100001110000100000000000
10	chiba	1100011110100011010011100010110000100000000000
11	sl{'q	11100111111100111101111000001110001100000000000
12	cv6&	11000111110110011011001001100000000100000000000

表 7 光通信における動作結果と目標

	通信精度	通信速度
目標	$FSR \geq 0.90$	100 bps 以上
動作結果	$FSR = 0.90$	143 bps

5 評価と考察

4.3 節で示した結果について評価と考察を行う。

5.1 光通信の評価と考察

光通信における動作結果と目標を表 7 に示す。通信精度、速度ともに目標に到達したため、光通信は有効であるといえる。

このような結果が得られた原因について考察する。本システムにおいて、光通信は LED と照度センサに覆うようにカゴを設置して、暗い環境で行った。これはカゴが無いと照明の影響を受けてしまい、照度センサが光の検出ができなかったためである。また、LED の明るさと照度センサの反応の強度のバランスを取ると、通信距離が 5 cm となった。これらの光を検知するのに適切な環境設置をしたことが、精度の目標に到達した原因だと考える。また、音通信では GI を設けたが、光通信ではこのような信号の間隔は開けなかった。これによって、必要最低限の信号の通信を行うことができ、速度が目標を超えたのだと考える。

5.2 音通信の評価と考察

音通信における動作結果と目標を表 8 に示す。通信精度、速度ともに目標には至らなかったため、音通信の有効性は低いといえる。

このような結果が得られた原因について考察する。精度について、目標に到達はできなかったが、誤差は 0.01 であった。4.2 節で述べた通り、類似研究では通信音量より雑音が大きいときに BER が 10 % (0.10) だった。今回の実験においても、他のグループの実験や会話があり、環境音はかなり影響があったと思われる。これらがスピーカからの音と干渉して正しい周波数を検出できなかったのだと考えられる。実験時に環境音と通信音量をそれぞれ測っておくことでこの問題の解決につながるだろう。また、特定の音だけ抜ける事があり、周波数によって検出しやすい・しにくい距離があったように考えられる。2000 ~ 3600 Hz の 1600 Hz の幅で同じように検出できる距離が見つからなかったことも原因だろう。次に、速度については、目標の 67 % の数値であった。これは、同期信号を 1 回ずつ出すところを急遽 2 回ずつ出すように変更したことや、終了の同期の安定のために GI を 1 つ増やしたことなどが原因として考えられる。これらをカバーするために音長を少しずつ短くして実験できれば、速度の改善が見られるだろう。また、別の課題として、Arduino の tone 関数を用いてスピーカから出る音があまり聞き心地が良くなかったことも挙げられる。通信を可聴化するという点において、聞き心地が良くないと通信の快適さが精神的な面で損なわれて

表 8 音通信における動作結果と目標

	通信精度	通信速度
目標	$BER < 0.10$	10 bps 以上
動作結果	$BER = 0.11$	6.7 bps

表 9 振動通信における動作結果と目標

	通信精度	通信速度
目標	$BER < 0.10$	10 bps 以上
動作結果	$BER = 0.20$	2.81 bps

しまう。仕組みは複雑になるが、他の音と組み合わせて和音にするなどの工夫が必要である。

5.3 振動通信の評価と考察

振動通信における動作結果と目標を表 9 に示す。通信精度、速度ともに目標には至らなかったため、音通信の有効性は低いといえる。

このような結果が得られた原因について考察する。精度について、結果は目標の 20 倍以上と大きく下回ってしまった。振動モータを高速でオンオフさせると、1 つ前の振動が残ってしまい、これが誤検知につながっている。これに加えて、加速度センサ自身が振動モータの振動の影響を受けてしまっているのでは無いかと考えた。次に速度について、結果は目標の 30 % 以下となった。振動でデータを送受信するとなると、データの表現方法に乏しく、4 進数に変化にして送信するといったようなデータの短縮や効率化が他のシステムに比べ困難である。振動モータの出力ではセンサに加える加速度が安定せず、これにより誤検知を軽減するために 1 回あたりの振動時間を長くしたことが原因だと考えた。また、課題として閾値を一度設定しても、土台を乗せる環境ごとに精度が変化し再設定する必要がある、どんな環境でも利用できるわけではないことが挙げられた。

5.4 システム全体の評価と考察

システム全体の目標として、精度について、3 部門における bit の一致率を 90 % として、これらの期待値 $0.90^3 = 0.729$ (72.9 %) を設定する。速度については、各部門における目標 bps が光で 100、音、振動で 10 であり、1 bit あたりの送信時間はこれらの逆数である。逆数の和から再度逆数を取ることで、全体の速度の目標を 4.76 bps 以上を設定する。

システム全体の精度は誤りビット数が 88 bit、総送信ビット数が 350 であるため、 $BER \approx 0.2514$ となる。よって、ビット一致率は $(1 - BER) \times 100 = 74.86\%$ である。ビット一致率とは別に文字としても一致率も求める。不一致の文字数が 34、総文字数が 50 であるため、文字不一致率が 68 %、つまり文字一致率は 32 % である。速度については総送信ビット数が 350、総通信時間の平均が 207.58 であるため、1.69 bps となる。

このような結果が得られた原因について考察する。精度については目標を上回ることができた。しかし、ビットが 70 % 一致していてもデコードされた文字をみて "chiba" が送信されたとは思えない

だろう。ここで文字一致率をみると 32%であった。これは最後に 2 進数に変換していることが原因だと考える。1 文字あたり、4 進数 4 桁のとき、2 進数では 8 桁が必要になり、1 桁あたりの重みは 2 進数の方が小さいため、1 桁誤ったときのビット（シンボル）一致率は 2 進数のほうが高くなる。しかし、どちらも 1 桁誤ると異なる文字にデコードされるもしくは文字化けすることは共通であり、桁数が多い分ビット一致率が高くても文字の一致率は 2 進数の方が低くなると考える。次に、速度について、目標の約 30%となった。光に比べて音と振動の速度がかなり遅く、これが速度の低下の原因である。本システムでは精度を向上させるために速度を抑えたが、これで文字一致率が 32%であることを考えると、通信速度に重点をおいてシステムを構築するほうが有効なシステムになると考えた。これらのことから、本システムは文字を伝送するシステムとしての有効性は低いと言える。

6 おわりに

本報告書では、通信を可視化、可聴化することを目的に、Arduino と LED、スピーカ、振動モータを用いた無線通信システムを開発した。具体的には、PC の文字入力から照度センサで LED の光を検知し、スピーカから出力された音をマイクでその周波数を検出し、加速度センサで振動モータの振動を判断して文字を伝送した。3 つの通信を結合したシステムでの通信は可能であったが、通信精度や通信速度の面では実用には程遠かった。加えて、どの通信も実施可能性や精度が環境に左右されやすく、自分たちで最適な環境を用意する必要があったことから、本システムは有効であるとはいえないということがわかった。通信可能な環境を増やしたり、センサの性能を最大限に引き出せば、有効なシステムになる可能性がある。現在通信できている環境での周囲の光や雑音の影響の調査をすることで、これらの問題の解決につながるだろう。

参考文献

- [1] 総務省,「電気通信事業者の平成 28 年熊本地震への対応状況」, 2016.https://www.soumu.go.jp/main_content/000432337.pdf
- [2] 総務省,「令和 6 年版 情報通信白書 第 1 部」, No.1-2, pp4-13, 2024.<https://www.soumu.go.jp/johotsusintokei/whitepaper/ja/r06/pdf/n1120000.pdf>
- [3] Rahul R. Sharma, Raunak, Akshay Sanganal, "Li-Fi Tecnology Transmission of data through light", Int.J.Computer Technology & Applications, Vol.5, No.1, 2014.
- [4] 松岡 保静, 音響データ通信技術: 音響 OFDM (<小特集>携帯情報機器における音響技術), 日本音響学会誌, Vol.68, No.3, pp.143-147, 2012.
- [5] 米澤 拓郎, 中澤 仁, 永田 智大, 徳田 英幸, 「Vinteraction: スマート端末のための振動を利用した情報送信インタラクション」, 情報処理学会論文誌, Vol.54, No.4, 1498-1506, 2013.
- [6] 総務省,「令和 4 年版 情報通信白書 第 2 部」, No.8-1, pp93-98, 2022.
- [7] Md. Zahid Hasan, Mohammad Reaz Hossain, Md. Ashraful Islam, Riaz Uddin Mondal, "Compara-

tive Study of Different Guard Time Intervals to Improve the BER Performance of Wimax Systems to Minimize the Effects of ISI and ICI under Adaptive Modulation Techniques over SUI-1 and AWGN Communication Channels", IJCSIS, Vol.6, No.2, pp128-132, 2009.

- [8] 毛利 哲也, 「シリーズ知能機械工学 6 デジタル信号処理」, 初版 2 刷, 共立出版株式会社, 2018.
- [9] "Receiver Minimum Input Sensitivity Test", IEEE Std 802.11-2020, Section 21.3.18.1 & Table 21-25.
- [10] 大石 智久, 「音楽性信号への符号化による携帯端末用音響通信」, 三重大学学術機関リポジトリ, 2014.

A Arduino のスケッチ

作成した Arduino のスケッチを以下に示す。

Listing 1 sound_tx.ino

```
1  const int maxmoji = 256 * 4; // 最大文字数256文字分 (ASCII) *4桁
2  int base4[maxmoji];          // 4進数の結果を格納する配列
3  int resultIndex = 0;
4  int time = 5;
5
6  void setup() {
7    Serial.begin(9600);
8    pinMode(A0, OUTPUT);
9    pinMode(A1, OUTPUT);
10   pinMode(A2, OUTPUT);
11   pinMode(A3, OUTPUT);
12   pinMode(A4, OUTPUT);
13 }
14
15 void loop() {
16   if (Serial.available() > 0) {
17     String input = Serial.readStringUntil('\n');
18     resultIndex = 0;
19
20     for (int i = 0; i < input.length(); i++) {
21
22       char c = input.charAt(i);
23       byte ascii = (byte)c;
24       getBase4(ascii);
25     }
26
27     Base4sousin();
28
29     for (int i = 0; i < resultIndex; i++) {
30       Serial.print(base4[i]);
31     }
32     Serial.println("");
33   }
34 }
35
36 void getBase4(byte b) {
37   int amari[4];
38   for (int i = 3; i >= 0; i--) {
39     amari[i] = b % 4;
```

```

40     b = b / 4;
41 }
42 for (int i = 0; i < 4; i++) {
43     base4[resultIndex++] = amari[i];
44 }
45 }
46
47 void Base4sousin() {
48     digitalWrite(A4, HIGH);
49
50     for (int k = 0; k < resultIndex; k++) {
51         int pin;
52         if (base4[k] == 0) {
53             pin = A0;
54         } else if (base4[k] == 1) {
55             pin = A1;
56         } else if (base4[k] == 2) {
57             pin = A2;
58         } else {
59             pin = A3;
60         }
61
62         digitalWrite(pin, HIGH);
63         delay(time); // この時間内に受信側が読み取り
64         digitalWrite(pin, LOW);
65     }
66
67     digitalWrite(A4, LOW); // 送信終了通知
68 }

```

Listing 2 sound_tx_itg.ino

```

1 #include "hack.h"
2
3 const int maxmoji = 256 * 4; // 最大文字数256文字分 (ASCII) *4桁
4 int base4[maxmoji]; // 4進数の結果を格納する配列
5 int resultIndex = 0; // 4進数変換したあとの文字数+1
6
7 // 音送信用の変数
8 const int speaker_pin = 3; // スピーカー接続ピン
9 unsigned long t = 300; // 音を鳴らす時間(ms)
10 unsigned long GI = t * 0.25; // GI長(tの25%)
11 unsigned int freq[16] = {2000, 2100, 2200, 2300, 2400, 2500, 2600, 2700, 2800, 2900,
    3000, 3100, 3200, 3300, 3400, 3500}; // 鳴らす周波数を設定 (最低周波数を開始ビット
    に)
12 unsigned int sfreq = 3600;
13 String str[16] = {"00", "01", "02", "03", "10", "11", "12", "13", "20", "21", "22", "23", "30", "

```

```

    31","32","33"} ; // 取得した4進数2桁
14 double txttime = 0;
15
16 // 光受信用の変数
17 const int detectPin = A4; // A4の光（送信中）を検出
18 bool inReceiving = false; // 現在受信中かどうか
19 const int read_time = 10; // 読み取る間隔[ms]
20
21 void setup() {
22     Serial.begin(9600);
23     pinMode(speaker_pin, OUTPUT);
24     for (int i = 0; i < 4; i++) {
25         pinMode(dataPins[i], INPUT);
26     }
27 }
28
29 void loop() {
30     int detectValue = analogRead(detectPin);
31     if (detectValue > threshold) {
32         // 送信中
33         if (!inReceiving) {
34             resultIndex = 0; // 初回のみ初期化
35             inReceiving = true;
36         }
37
38         int val = detectBitByDuration(read_time); // msごとの読み取り
39         if (val != -1) {
40             base4[resultIndex++] = val; // 桁数を数える
41         }
42     }else{
43         // 送信終了時
44         if (inReceiving) {
45             txttime = 0;
46             txttime = millis();
47             String s = "";
48
49             // 開始用の周波数を出力
50             syncTone(speaker_pin, sfreq, t, GI);
51             syncTone(speaker_pin, sfreq, t, GI);
52
53             for (int i=0; i<resultIndex; i++){
54                 s += String(base4[i]);
55             }
56             for (int k=0; k<resultIndex; k+=2){
57                 // 前から2桁ずつ音を鳴らす
58                 for (int l=0; l<16; l++){
59                     if(s.substring(k, k+2) == str[l]){
60                         tone(speaker_pin, freq[l]);

```

```

61         delay(t);
62         noTone(speaker_pin);
63         delay(GI);
64     }
65 }
66 }
67 delay(GI);
68
69 // 終了用の周波数を出力
70 syncTone(speaker_pin, sfreq, t, GI);
71 syncTone(speaker_pin, sfreq, t, GI);
72
73 // 確認用シリアル表示（読み取り）
74 Serial.print("受信データ:");
75 for (int m=0; m<resultIndex; m++) {
76     Serial.print(base4[m]);
77 }
78 Serial.println("");
79 Serial.println("-----");
80 Serial.print("Base4:");
81 for (int i = 0; i < resultIndex; i++) {
82     Serial.print(base4[i]); // 4進数の左から表示
83     s += String(base4[i]);
84 }
85 Serial.println("");
86 for (int l=0; l<resultIndex; l+=2){
87     for (int m=0; m<16; m++){
88         if(s.substring(l, l+2) == str[m]){
89             String foo = s.substring(l, l+2);
90             int bar = ((foo.charAt(0) - '0') * 4 + (foo.charAt(1) - '0'));
91             Serial.print(freq[bar]);
92             Serial.println("_Hz");
93         }
94     }
95 }
96 Serial.println("-----");
97 inReceiving = false; // フラグを戻す
98 }
99 }
100 }

```

Listing 3 hack.h

```

1  #ifndef HACK_H
2  #define HACK_H
3
4  #include <Arduino.h>

```

```

5
6  const int dataPins[4] = {A0, A1, A2, A3};
7  const int threshold = 50;
8
9  int detectBitByDuration(int durationMs);
10 void syncTone(int pin, unsigned int frequency, unsigned long delay, unsigned long GI)
    ;
11
12 #endif

```

Listing 4 hack.cpp

```

1  #include "hack.h"
2
3  int detectBitByDuration(int durationMs) {
4      int lightDetected[4] = {0, 0, 0, 0};
5      unsigned long start = millis();
6
7      while (millis() - start < durationMs) {
8          for (int i = 0; i < 4; i++) {
9              if (analogRead(dataPins[i]) > threshold) { // 光を検出したらカウント
10                 lightDetected[i]++;
11             }
12         }
13     }
14
15     // 最も多く光を検出したピンを選ぶ（ノイズ対策）
16     int maxIndex = -1;
17     int maxVal = 0;
18     for (int i = 0; i < 4; i++) {
19         if (lightDetected[i] > maxVal) {
20             maxVal = lightDetected[i];
21             maxIndex = i;
22         }
23     }
24
25     // しきい値以上の光検出があれば有効
26     if (maxVal > 3) {
27         return maxIndex;
28     } else {
29         return -1; // ノイズとみなして無視
30     }
31 }
32
33 void syncTone(int pin, unsigned int frequency, unsigned long delay, unsigned long GI)
    ){
34     tone(pin, frequency);

```

```

35     delay(delayt);
36     noTone(pin);
37     delay(GI);
38 }

```

Listing 5 vibration_tx_itg.ino

```

1  #include <arduinoFFT.h>
2
3  // --- 受信側（マイク）設定 ---
4  const uint16_t samples = 256;           // FFTサンプル数
5  const double samplingFrequency = 8000.0; // サンプル周波数 (Hz)
6  const double samplingIntervalUs = 1000000.0 / samplingFrequency; // サンプル間隔
   (マイクロ秒)
7  const uint8_t micPin = A0;              // マイク入力用アナログピン
8
9  double vReal[samples];
10 double vImag[samples];
11
12 ArduinoFFT FFT = ArduinoFFT(vReal, vImag, samples, samplingFrequency);
13
14 String decodedData = "";
15 bool receiving = false;
16 int startSignalCount = 0;
17 int endSignalCount = 0;
18
19 // --- 受信側データマッピング ---
20 String str[16] = {"00", "01", "02", "03", "10", "11", "12", "13", "20", "21", "22", "23", "30", "
   31", "32", "33"};
21 unsigned int binaryarray[] = {0, 1, 10, 11, 100, 101, 110, 111, 1000, 1001, 1010,
   1011, 1100, 1101, 1110, 1111};
22 const int maxlit = 256 * 4; // 受信データ格納用配列の最大サイズ
23 int binary[maxlit]; // パディングされた2進数を整数として格納（主にデバッグ表示用）
24 int ReceivedLength;
25
26 // --- 送信側（振動モーター）設定 ---
27 const int motorPin = 5; // 振動モーター用デジタルピンをD5に設定
28 const unsigned long bitDuration = 200; // 1ビットの持続時間 (ms)
29 const int signalBitCount = 7; // 送信開始信号ビット数（1を7回）
30 const int endSignalBitCount = 10; // 送信終了信号ビット数（0を10回）
31
32 // --- 関数プロトタイプ宣言 ---
33 // 受信側
34 double detectFrequency();
35 int decodeFrequency(double freq);
36 // 受信した4進数文字列をASCIIバイト列に変換し、各バイトを振動送信する関数
37 void convertAndSendVibration(String quadStr);

```

```

38
39 // 送信側
40 void sendVibrationBit(int bitValue);
41 void sendByte(byte data);
42 void sendStartSignal();
43 void sendEndSignal();
44
45
46 void setup() {
47     Serial.begin(9600);           // シリアル通信の初期化
48     pinMode(motorPin, OUTPUT);    // モーターピンを出力に設定
49     digitalWrite(motorPin, LOW);  // モーターを初期状態はOFFにする
50 }
51
52 void loop() {
53     // --- 受信処理ブロック ---
54     double freq = detectFrequency();
55     int decoded = decodeFrequency(freq);
56     int binaryIndex = 0; // binary配列の書き込みインデックスをここで宣言・初期化
57     for (int i = 0; i < maxlit; i++) {
58         binary[i] = 0; // binary配列を毎回リセット
59     }
60
61     if (decoded == -1) { // 開始/終了シグナル
62         if (!receiving) {
63             // 通信開始前
64             startSignalCount++;
65             if (startSignalCount >= 2) { // 2回以上受け取ったら開始
66                 decodedData = ""; // 開始時に初期化
67                 receiving = true;
68                 startSignalCount = 0;
69                 endSignalCount = 0;
70                 Serial.println("-----_Start_receiving_-----");
71             } else {
72                 Serial.print("Start_signal_received_");
73                 Serial.print(startSignalCount);
74                 Serial.println("_time(s)._Waiting_for_2nd_signal...");
75             }
76         } else {
77             // 通信中
78             endSignalCount++;
79             if (endSignalCount >= 2) { // 2回以上受け取ったら終了
80                 receiving = false;
81                 startSignalCount = 0;
82                 endSignalCount = 0;
83                 Serial.println("-----_End_receiving_-----");
84                 Serial.print("Received(row):_");
85                 Serial.println(decodedData);

```

```

86
87 Serial.print("Decoded_ASCII:_"); // 新しいラベル
88 String fullBinaryStringForDisplay = "";
89 int currentReceivedLengthForDisplay = decodedData.length();
90
91 for(int i = 0; i < currentReceivedLengthForDisplay; i += 2){
92     String currentQuad = decodedData.substring(i, i + 2);
93     bool found = false;
94
95     for(int j = 0; j < 16; j++){
96         if(currentQuad == str[j]){
97             String binStr = String(binaryarray[j]);
98             while (binStr.length() < 4) {
99                 binStr = "0" + binStr;
100             }
101             fullBinaryStringForDisplay += binStr;
102             found = true;
103             break;
104         }
105     }
106 }
107
108 // 8ビットごとにASCII文字に変換して表示
109 for (int i = 0; i + 7 < fullBinaryStringForDisplay.length(); i += 8) {
110     String byteBinaryStr = fullBinaryStringForDisplay.substring(i, i + 8);
111     long byteValue = 0;
112     for (int k = 0; k < 8; k++) {
113         if (byteBinaryStr.charAt(k) == '1') {
114             byteValue += (1 << (7 - k));
115         }
116     }
117     Serial.print((char)byteValue); // 文字を直接表示
118 }
119 Serial.println(); // デコードされた文字の表示後に改行
120
121 // 受信した4進数データをASCIIに変換し、そのバイナリ表現を振動として送信
122 convertAndSendVibration(decodedData);
123
124 // --- 以下はデバッグ表示用（受信機のデバッグ出力として残す） ---
125 ReceivedLength = decodedData.length();
126 Serial.print("Received_length(row):_");
127 Serial.println(ReceivedLength);
128
129 for(int i=0; i<ReceivedLength; i+=2){
130     String currentQuad = decodedData.substring(i, i+2);
131     for(int j=0; j<16; j++){
132         if(currentQuad == str[j]){
133             if(binaryIndex < (sizeof(binary)/sizeof(binary[0]))){

```



```

134         String paddedBinaryStr = String(binaryarray[j]);
135         while (paddedBinaryStr.length() < 4) {
136             paddedBinaryStr = "0" + paddedBinaryStr;
137         }
138         binary[binaryIndex] = paddedBinaryStr.toInt();
139         Serial.print("binaryarray_(padded_int):_");
140         Serial.println(binary[binaryIndex]);
141         binaryIndex++;
142     }else{
143         Serial.println("Error:_Storage_overflow_for_binary_array.");
144     }
145     for(int l=0; l<binaryIndex; l++){
146         Serial.print(binary[l]);
147     }
148     Serial.println("");
149     Serial.print("binary_(original_value):_");
150     Serial.println(binaryarray[j]);
151     break;
152 }
153 }
154 }
155 Serial.print("finally_binary[_]_contents_(padded_int):_["");
156 for (int i = 0; i < binaryIndex; i++) {
157     Serial.print(binary[i]);
158     if (i < binaryIndex - 1) {
159         Serial.print(",_");
160     }
161 }
162 Serial.println("]");
163 // -----
164
165 } else {
166     Serial.print("End_signal_reveived_");
167     Serial.print(endSignalCount);
168     Serial.println("_time(s)._Waiting_for_2nd_signal_to_end...");
169 }
170 }
171 } else if (receiving && decoded >= 0 && decoded <= 15) {
172     // データ受信中
173     endSignalCount = 0; // データが来たら終了カウントをリセット
174     char quadDigit1 = '0'+(decoded/4);
175     char quadDigit2 = '0'+(decoded%4);
176     String quadStr = String(quadDigit1) + String(quadDigit2);
177     Serial.print("Adding_quad:_");
178     Serial.println(quadStr);
179     decodedData += quadStr;
180 } else if (!receiving && decoded >= 0 && decoded <= 15) {
181     // 通信開始前にデータが来ても無視し、開始カウントをリセット

```

```

182     startSignalCount = 0;
183 }
184 delay(290); // 次の音まで待機
185 }
186
187 // --- 受信側関数定義 ---
188
189 // FFTでピーク周波数を検出する関数
190 double detectFrequency() {
191     unsigned long targetTime = micros();
192     for (int i = 0; i < samples; i++) {
193         while (micros() < targetTime) {}
194         vReal[i] = analogRead(micPin);
195         vImag[i] = 0;
196         targetTime += samplingIntervalUs;
197     }
198
199     // DCオフセット除去
200     double mean = 0;
201     for (int i = 0; i < samples; i++) {
202         mean += vReal[i];
203     }
204     mean /= samples;
205     for (int i = 0; i < samples; i++) {
206         vReal[i] -= mean;
207     }
208
209     FFT.windowing(FFT_WIN_TYP_HAMMING, FFT_FORWARD);
210     FFT.compute(FFT_FORWARD);
211     FFT.complexToMagnitude();
212
213     double peak = 0;
214     int peakIndex = 0;
215     for (int i = 1; i < samples / 2; i++) {
216         if (vReal[i] > peak) {
217             peak = vReal[i];
218             peakIndex = i;
219         }
220     }
221     return (peakIndex * samplingFrequency) / samples;
222 }
223
224 // 周波数から4進数2桁データをデコード (0~15)
225 int decodeFrequency(double freq) {
226     int freqInt = (int)(freq + 0.5);
227
228     if (freqInt >= 3570 && freqInt <= 3630) return -1; // 開始/終了信号
229

```

```

230     for (int i = 0; i < 16; i++) {
231         int expected = 2000 + 100 * i;
232         if (freqInt >= expected - 49 && freqInt <= expected + 49) {
233             return i;
234         }
235     }
236     return -2; // 無効な周波数
237 }
238
239 // 受信した4進数文字列をASCIIバイト列に変換し、各バイトを振動送信する関数
240 void convertAndSendVibration(String quadStr) {
241     String fullBinaryString = "";
242     int currentReceivedLength = quadStr.length();
243
244     // 4進数チャンクをパディングされた4ビットの2進数文字列に変換して連結
245     for(int i = 0; i < currentReceivedLength; i += 2){
246         String currentQuad = quadStr.substring(i, i + 2);
247         bool found = false;
248
249         for(int j = 0; j < 16; j++){
250             if(currentQuad == str[j]){
251                 String binStr = String(binaryarray[j]);
252                 while (binStr.length() < 4) {
253                     binStr = "0" + binStr;
254                 }
255                 fullBinaryString += binStr;
256                 found = true;
257                 break;
258             }
259         }
260         if (!found) {
261             Serial.print("警告:_未知の4進数チャンク_");
262             Serial.print(currentQuad);
263             Serial.println("'_が検出されました。スキップして_'0000'_'を挿入します。");
264             fullBinaryString += "0000"; // 未知のチャンクの場合、デフォルトで0000を挿入
265         }
266     }
267
268     Serial.print("_Binary_string_(for_vibration):_");
269     Serial.println(fullBinaryString);
270     Serial.print("_Converted_ASCII_(for_vibration):_");
271     Serial.println("");
272
273     // まず送信開始信号を送る
274     sendStartSignal();
275     delay(bitDuration); // 送信開始信号後の待機
276
277     // 8ビットごとにASCII文字に変換し、そのバイトを振動送信

```

```

278     for (int i = 0; i + 7 < fullBinaryString.length(); i += 8) {
279         String byteBinaryStr = fullBinaryString.substring(i, i + 8);
280         long byteValue = 0;
281
282         for (int k = 0; k < 8; k++) {
283             if (byteBinaryStr.charAt(k) == '1') {
284                 byteValue += (1 << (7 - k));
285             }
286         }
287         Serial.print((char)byteValue); // シリアルモニターに文字を表示 (ASCII)
288
289         // デコードされたバイトを振動として送信
290         sendByte((byte)byteValue);
291     }
292     Serial.println();
293
294     // 全てのバイト送信後に送信終了信号を送る
295     sendEndSignal();
296     Serial.println("Vibration_transmission_completed");
297     digitalWrite(motorPin, LOW); // モーターをOFFにする
298 }
299
300
301 // --- 送信側関数定義 ---
302
303 // 1ビットの値を振動として出力する関数
304 void sendVibrationBit(int bitValue) {
305     digitalWrite(motorPin, bitValue == 1 ? HIGH : LOW);
306     delay(bitDuration);
307     digitalWrite(motorPin, LOW); // 振動のOFFを明示
308 }
309
310 // 1バイトのデータをビット列に分解して振動として送信する関数
311 void sendByte(byte data) {
312     Serial.print("__Bits_for_");
313     Serial.print((char)(data & 0x7F)); // 送信する文字 (7ビットASCII) を表示
314     Serial.print(":_");
315     for (int bit = 6; bit >= 0; bit--) { // 7ビットASCIIの範囲でループ (MSBからLSBへ)
316         int bitValue = (data >> bit) & 0x01; // ビット値を取得
317         sendVibrationBit(bitValue);           // 振動として送信
318         Serial.print(bitValue);               // シリアルモニターにビット値を表示
319     }
320     Serial.println();
321 }
322
323 // 送信開始信号を送る関数
324 void sendStartSignal() {
325     Serial.println("---_振動送信開始信号_---"); // シリアルモニターに表示

```

```

326     for (int i = 0; i < signalBitCount; i++) {
327         sendVibrationBit(1); // 1 (HIGH) を signalBitCount 回送信
328     }
329 }
330
331 // 送信終了信号を送る関数
332 void sendEndSignal() {
333     Serial.println("---_振動送信終了信号_---"); // シリアルモニターに表示
334     for (int i = 0; i < endSignalBitCount; i++) {
335         sendVibrationBit(0); // 0 (LOW) を endSignalBitCount 回送信
336     }
337 }

```

Listing 6 vibration_rcv.ino

```

1
2 const int xPin = A0;
3 const int yPin = A1;
4 const int zPin = A2;
5
6 const unsigned long bitDuration = 230; // 1ビットの持続時間 (ms)
7 const float vibrationThreshold = 12.9; // 振動検出のしきい値
8 const int signalBitCount = 5; // 開始信号 (1*7回)
9 const int endSignalBitCount = 11; // 終了信号 (0*10回)
10 const int maxBits = 512; // 最大ビット数バッファ
11
12 // 加速度センサから3軸データを読み取る
13 void readXYZ(int &x, int &y, int &z) {
14     x = analogRead(xPin);
15     y = analogRead(yPin);
16     z = analogRead(zPin);
17 }
18
19 // 指定時間内に振動があったかを検出
20 bool detectVibration(unsigned long duration) {
21     unsigned long start = millis();
22     int baseX, baseY, baseZ;
23     readXYZ(baseX, baseY, baseZ);
24
25     int vibrationCount = 0;
26     int samples = 0;
27
28     while (millis() - start < duration) {
29         int xNow, yNow, zNow;
30         readXYZ(xNow, yNow, zNow);
31         float delta = sqrt(
32             pow(xNow - baseX, 2) +

```

```

33     pow(yNow - baseY, 2) +
34     pow(zNow - baseZ, 2)
35 );
36 if (delta > vibrationThreshold) vibrationCount++;
37 samples++;
38 delay(5); // 過剰サンプリング防止
39 }
40
41 return (vibrationCount > samples / 2);
42 }
43
44 // 開始信号 (1*7) を検出
45 bool detectStartSignal() {
46     Serial.println("Checking_start_signal...");
47     for (int i = 0; i < signalBitCount; i++) {
48         if (!detectVibration(bitDuration)) {
49             Serial.println("Start_signal_failed");
50             return false;
51         }
52         Serial.print("1");
53     }
54     Serial.println("\n[Start_Detected]");
55     return true;
56 }
57
58 // 終了信号 (最後の10ビットが0) を検出
59 bool detectEndSignal(const bool *bits, int len) {
60     if (len < endSignalBitCount) return false;
61     for (int i = len - endSignalBitCount; i < len; i++) {
62         if (bits[i]) return false;
63     }
64     return true;
65 }
66
67 // --- 初期化 ---
68 void setup() {
69     Serial.begin(9600);
70     pinMode(xPin, INPUT);
71     pinMode(yPin, INPUT);
72     pinMode(zPin, INPUT);
73     Serial.println("Ready_to_receive_vibration-based_communication");
74 }
75
76 // --- メインループ ---
77 void loop() {
78     static bool bitsBuffer[maxBits];
79     static int bitsCount = 0;
80

```

```

81 Serial.println("Waiting_for_start_signal...");
82 while (!detectStartSignal()); // 開始信号を待機
83
84 delay(3*bitDuration);
85
86 bitsCount = 0;
87 unsigned long bitStart;
88
89 // --- ビット受信ループ ---
90 while (true) {
91     bitStart = millis();
92     bool bit = detectVibration(bitDuration);
93     bitsBuffer[bitsCount++] = bit;
94     Serial.print(bit ? '1' : '0');
95
96     // 終了信号検出
97     if (bitsCount >= endSignalBitCount && detectEndSignal(bitsBuffer, bitsCount)) {
98         Serial.println("\n[End_Detected]");
99         break;
100     }
101
102     // ビット数上限
103     if (bitsCount >= maxBits) {
104         Serial.println("\n[Error]_Buffer_overflow._Resetting.");
105         return;
106     }
107
108     // 次のビットまで待機 (正確な周期維持)
109     while (millis() - bitStart < bitDuration) {
110         delay(1);
111     }
112 }
113
114 // --- 7ビットASCIIに復元 ---
115 int dataBits = bitsCount - endSignalBitCount;
116 int usableBits = (dataBits / 7) * 7;
117
118 Serial.print("Decoded_message:_");
119 for (int i = 0; i + 6 < usableBits; i += 7) {
120     byte c = 0;
121     for (int b = 0; b < 7; b++) {
122         if (bitsBuffer[i + b]) {
123             c |= (1 << (6 - b)); // MSBからLSB順に復元
124         }
125     }
126     Serial.print((char)c);
127 }
128 Serial.println();

```

